

Question 1. Right Half Pyramid

Pattern

```
*
**
***
****
*****
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to **rows**.
3. Run the inner loop from 1 to **i**.
4. Print one ***** in each inner loop iteration.
5. Print a new line after every row.

Code

```
#include <iostream>
using namespace std;
```

```
int main() {
    int rows;
    cin >> rows;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            cout << " ",
        }
        cout << endl;
    }

    return 0;
}
```

Condition Explanation

Outer Loop

```
for(int i = 1; i <= rows; i++)
```

- **i = 1** → First row.
- **i <= rows** → Runs until the last row.

- **i++** → Moves to the next row.

Inner Loop

```
for(int j = 1; j <= i; j++)
```

- **j = 1** → Starts printing the first star.
- **j <= i** → Prints stars equal to the current row number.
- **j++** → Prints one more star each iteration.

New Line

```
cout << endl;
```

Moves the cursor to the next row after printing all stars.

Complexity

- **Time:** $O(n^2)$
- **Space:** $O(1)$

Question 2. Inverted Right Half Pyramid

Pattern

```
****
***
**
*

```

Algorithm

1. Input the number of rows.
2. Run the outer loop from **rows** to 1.
3. Run the inner loop from 1 to **i**.
4. Print one ***** in each iteration of the inner loop.
5. Print a new line after every row.

Code

```
#include <iostream>
using namespace std;
```

Pattern

```
*  
  
**  
  
***  
  
****  
  
*****
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to rows.
3. Print rows - i spaces.
4. Print i stars.
5. Move to the next line.

Code

```
#include <iostream>  
using namespace std;
```

```
int main() {
```

```
    int rows;
```

```
    cin >> rows;
```

```
    for (int i = 1; i <= rows; i++) {
```

```
        for (int j = 1; j <= rows - i; j++) {  
            cout << " ";  
        }  
    }
```

```
    for (int j = 1; j <= i; j++) {
```

```
int main() {  
    int rows;  
    cin >> rows;  
  
    for (int i = rows; i >= 1; i--) {  
        for (int j = 1; j <= i; j++) {  
            cout << "star",  
        }  
  
        cout << endl;  
    }  
  
    return 0;  
}
```

Condition Explanation

for(int i = rows; i >= 1; i--)

- i = rows → Starts from the last row (maximum stars).
- i >= 1 → Continues until one row remains.
- i-- → Decreases one row after each iteration.

for(int j = 1; j <= i; j++)

- j = 1 → Starts printing the first star.
- j <= i → Prints stars equal to the current row number.
- Since i decreases every row, the number of stars also decreases.
- j++ → Prints the next star.

cout << endl;

Moves the cursor to the next line after printing all stars of the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Question 3. Left Half Pyramid

```
cout << " ";
}
```

```
cout << endl;
}
```

```
return 0;
}
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- $i = 1 \rightarrow$ Starts from the first row.
- $i \leq rows \rightarrow$ Runs until the last row.
- $i++ \rightarrow$ Moves to the next row.

```
for(int j = 1; j <= rows - i; j++)
```

- Prints spaces before the stars.
- $rows - i$ decreases every row.
- First row gets the maximum spaces.
- Last row gets 0 spaces.
- $j++$ prints one space in each iteration.

```
for(int j = 1; j <= i; j++)
```

- Prints stars after the spaces.
- i increases every row.
- First row prints 1 star.
- Last row prints $rows$ stars.
- $j++$ prints one star in each iteration.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Question 4. Inverted Left Half Pyramid

Pattern

```
*****
*****
****
***
**
*
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to $rows$.
3. Print $i - 1$ spaces.
4. Print $rows - i + 1$ stars.
5. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i - 1; j++) {
            cout << " ";
        }
        for (int j = 1; j <= rows - i + 1; j++) {
            cout << "*";
        }
    }
}
```

```

        cout << endl;
    }

    return 0;
}

```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- $i = 1 \rightarrow$ Starts from the first row.
- $i \leq \text{rows} \rightarrow$ Runs until the last row.
- $i++ \rightarrow$ Moves to the next row.

```
for(int j = 1; j <= i - 1; j++)
```

- Prints spaces before the stars.
- $i - 1$ increases every row.
- First row gets 0 spaces.
- Last row gets the maximum spaces.
- $j++$ prints one space in each iteration.

```
for(int j = 1; j <= rows - i + 1; j++)
```

- Prints stars after the spaces.
- $\text{rows} - i + 1$ decreases every row.
- First row prints the maximum stars.
- Last row prints only 1 star.
- $+1$ is added because counting starts from 1, not 0.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print increasing spaces first, then decreasing stars in every row.

Question 5. Full Pyramid

Pattern

```

*
***
*****
*****
*****

```

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to rows.
3. Print $\text{rows} - i$ spaces.
4. Print $2 \times i - 1$ stars.
5. Move to the next line.

Code

```
#include <iostream>
using namespace std;
```

```
int main() {
    int rows;
    cin >> rows;
```

```
    for (int i = 1; i <= rows; i++) {
```

```
        for (int j = 1; j <= rows - i; j++) {
            cout << " ";
        }
```

```
        for (int j = 1; j <= (2 * i - 1); j++) {
            cout << "**";
        }
```

```
        cout << endl;
    }
```

```
    return 0;
}
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

Question 6. Inverted Full Pyramid

Pattern

```
*****
*****
*****
****
***
**
*
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from **rows** to **1**.
3. Print **rows - i** spaces.
4. Print **2 * i - 1** stars.
5. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = rows; i >= 1; i--) {
        for (int j = 1; j <= rows - i; j++) {
            cout << " ";
        }

        for (int j = 1; j <= (2 * i - 1); j++) {
            cout << "**";
        }

        cout << endl;
    }

    return 0;
}
```

Condition Explanation

for(int i = rows; i >= 1; i--)

- **i = 1** → Starts from the first row.
- **i <= rows** → Runs until the last row.
- **i++** → Moves to the next row.

```
for(int j = 1; j <= rows - i; j++)
```

- Prints spaces before the stars.
- **rows - i** decreases every row.
- First row gets the maximum spaces.
- Last row gets **0** spaces.

```
for(int j = 1; j <= (2 * i - 1); j++)
```

- Prints stars after the spaces.
- **2 * i - 1** gives an odd number of stars.
- Each new row adds **2** more stars.
- Row 1 → **1** star.
- Row 2 → **3** stars.
- Row 3 → **5** stars.
- Row 4 → **7** stars.
- Row 5 → **9** stars.

Why **2 * i - 1**?

- A full pyramid always contains an **odd number of stars**.
- Multiplying by **2** increases the count by **2** every row.
- Subtracting **1** makes the first row start with **1** star.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: **O(n²)**
- Space Complexity: **O(1)**

Logic in One Line

Print decreasing spaces first, then print an odd number of stars (**2*i-1**) in every row.

- $i = \text{rows}$ → Starts from the last row (maximum stars).
- $i >= 1$ → Continues until the first row.
- $i--$ → Moves upward by decreasing one row each time.

```
for(int j = 1; j <= rows - i; j++)
```

- Prints spaces before the stars.
- $\text{rows} - i$ increases every row.
- First row gets 0 spaces.
- Last row gets the maximum spaces.

```
for(int j = 1; j <= (2 * i - 1); j++)
```

- Prints stars after the spaces.
- $2 * i - 1$ gives an odd number of stars.
- Since i decreases every row, the number of stars also decreases by 2.
- Row 5 → 9 stars.
- Row 4 → 7 stars.
- Row 3 → 5 stars.
- Row 2 → 3 stars.
- Row 1 → 1 star.

Why $2 * i - 1$?

- A full pyramid always contains an odd number of stars.
- As i decreases, the star count decreases automatically.
- This produces a perfectly inverted pyramid.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print increasing spaces first, then print a decreasing odd number of stars
 $(2 * i - 1)$ in every row.

Question 7. Hollow Square

Pattern

```
*****
* *
* *
* *
* *
*****
```

Algorithm

1. Input the size of the square.
2. Run the outer loop from 1 to rows .
3. Run the inner loop from 1 to rows .
4. Print * if the current position is on the boundary.
5. Otherwise, print a space.
6. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= rows; j++) {
            if (i == 1 || i == rows || j == 1 || j == rows) {
                cout << " * ";
            } else {
                cout << " ";
            }
        }
        cout << endl;
    }
    return 0;
}
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- Controls the rows.
- `i = 1` → First row.
- `i <= rows` → Last row.
- `i++` → Moves to the next row.

```
for(int j = 1; j <= rows; j++)
```

- Controls the columns.
- `j = 1` → First column.
- `j <= rows` → Last column.
- `j++` → Moves to the next column.

```
if(i == 1 || i == rows || j == 1 || j == rows)
```

- `i == 1` → First row.
- `i == rows` → Last row.
- `j == 1` → First column.
- `j == rows` → Last column.
- `||` (OR) means if **any one** of these conditions is true, print `*`.
- These four conditions together form the square's boundary.

else

- Executes only for the inside part of the square.
- Prints spaces to make the pattern hollow.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print `*` only on the boundary (first row, last row, first column, and last column);
print spaces everywhere else.

Question 8. Hollow Rectangle

Pattern

```
*****
*   *
*   *
*   *
*****
```

Algorithm

1. Input the number of rows and columns.
2. Run the outer loop from `1` to `rows`.
3. Run the inner loop from `1` to `cols`.
4. Print `*` if the current position is on the boundary.
5. Otherwise, print a space.
6. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows, cols;
    cin >> rows >> cols;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= cols; j++) {
            if (i == 1 || i == rows || j == 1 || j == cols) {
                cout << " * ";
            } else {
                cout << " ";
            }
        }
        cout << endl;
    }

    return 0;
}
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- Controls the rows.
- `i = 1` → First row.
- `i <= rows` → Last row.
- `i++` → Moves to the next row.

```
for(int j = 1; j <= cols; j++)
```

- Controls the columns.
- `j = 1` → First column.
- `j <= cols` → Last column.
- `j++` → Moves to the next column.

```
if(i == 1 || i == rows || j == 1 || j == cols)
```

- `i == 1` → First row.
- `i == rows` → Last row.
- `j == 1` → First column.
- `j == cols` → Last column.
- If **any one** of these conditions is true, print `*`.
- These conditions create the rectangle's border.

```
else
```

- Executes for the inside part of the rectangle.
- Prints spaces to make the rectangle hollow.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(\text{rows} \times \text{cols})$
- Space Complexity: $O(1)$

Logic in One Line

Print `*` only on the boundary (first row, last row, first column, and last column);
print spaces inside to create a hollow rectangle.

Question 9. Hollow Right Half Pyramid

Pattern

```
*  
**  
**  
* *  
*****
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from `1` to `rows`.
3. Print `rows - i` spaces.
4. Run the inner loop from `1` to `i`.
5. Print `*` only on the boundary (first column, last column, or last row).
6. Otherwise, print a space.
7. Move to the next line.

Code

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int rows;  
    cin >> rows;  
  
    for (int i = 1; i <= rows; i++) {  
  
        for (int j = 1; j <= rows - i; j++) {  
            cout << " ";  
        }  
  
        for (int j = 1; j <= i; j++) {  
  
            if (j == 1 || j == i || i == rows) {  
                cout << "*";  
            } else {  
                cout << " ";  
            }  
  
        }  
  
        cout << endl;  
    }  
}
```


- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print leading spaces, then print $*$ only on the left boundary, right boundary, and bottom row; print spaces inside to make the pyramid hollow.

Question 10. Hollow Inverted Right Half Pyramid

Pattern

```
*****
* *
* *
* *
**
*
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from $rows$ to 1 .
3. Run the inner loop from 1 to i .
4. Print $*$ only on the boundary (first column, last column, or first row).
5. Otherwise, print a space.
6. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = rows; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            if (j == 1 || j == i || i == rows) {
                cout << " * ";
            } else {
                cout << " ";
            }
        }
    }
}
```

```
return 0;
}
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- Controls the rows.
- $i = 1 \rightarrow$ First row.
- $i \leq rows \rightarrow$ Last row.
- $i++ \rightarrow$ Moves to the next row.

```
for(int j = 1; j <= rows - i; j++)
```

- Prints spaces before the pyramid.
- $rows - i$ decreases every row.
- First row gets maximum spaces.
- Last row gets 0 spaces.

```
for(int j = 1; j <= i; j++)
```

- Prints the stars (or spaces) of the current row.
- Number of positions increases with each row.

```
if(j == 1 || j == i || i == rows)
```

- $j = 1 \rightarrow$ Prints the left boundary.
- $j = i \rightarrow$ Prints the right boundary.
- $i == rows \rightarrow$ Prints the complete bottom row.
- If none of these conditions are true, print a space to make the pyramid hollow.

```
else
```

- Prints spaces inside the pyramid.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

Logic in One Line

Print `*` on the top row, left boundary, and right boundary; print spaces inside to create a hollow inverted right half pyramid.

Question 11. Hollow Full Pyramid

Pattern

```
*
* *
* *
* *
* *
*****
```

Algorithm

- 1. Input the number of rows.
- 2. Run the outer loop from `1` to `rows`.
- 3. Print `rows - i` spaces.
- 4. Run the inner loop from `1` to `2*i-1`.
- 5. Print `*` only on the first position, last position, or last row.
- 6. Otherwise, print a space.
- 7. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = 1; i <= rows; i++) {
        // Spaces
        for (int j = 1; j <= rows - i; j++) {
            cout << " ";
        }

        // Stars and Inner Spaces
        for (int j = 1; j <= (2 * i - 1); j++) {
            if (j == 1 || j == (2 * i - 1) || i == rows) {
                cout << "*";
            } else {
                cout << " ";
            }
        }
    }
}
```

```
}
    cout << endl;
}

return 0;
}
```

Condition Explanation

for(int i = rows; i >= 1; i--)

- Controls the rows.
- `i = rows` → Starts from the first row (maximum stars).
- `i >= 1` → Continues until the last row.
- `i--` → Decreases one row in each iteration.

for(int j = 1; j <= i; j++)

- Prints the positions of the current row.
- Number of positions decreases every row.

if(j == 1 || j == i || i == rows)

- `j = 1` → Prints the left boundary.
- `j = i` → Prints the right boundary.
- `i = rows` → Prints the complete top row.
- Otherwise, only the boundary stars are printed.

else

- Prints spaces inside the pyramid to make it hollow.

cout << endl;

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

```
        cout << " ";
    }
}
```

```
}
```

```
        cout << endl;
    }
}
```

```
return 0;
}
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- Controls the rows.
- $i = 1 \rightarrow$ First row.
- $i \leq \text{rows} \rightarrow$ Last row.
- $i++ \rightarrow$ Moves to the next row.

```
for(int j = 1; j <= rows - i; j++)
```

- Prints leading spaces.
- $\text{rows} - i$ decreases every row.
- First row gets maximum spaces.
- Last row gets 0 spaces.

```
for(int j = 1; j <= (2*i-1); j++)
```

- Prints the positions of the pyramid.
- $2*i-1$ gives an odd number of positions.
- Every row increases by 2 positions.

```
if(i == 1 || j == (2*i-1) || i == rows)
```

- $j == 1 \rightarrow$ Prints the left boundary.
- $j == (2*i-1) \rightarrow$ Prints the right boundary.
- $i == \text{rows} \rightarrow$ Prints the complete bottom row.
- Otherwise, only boundary stars are printed.

```
else
```

- Prints spaces inside the pyramid.
- Makes the pyramid hollow.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print leading spaces, then print $*$ on the left boundary, right boundary, and bottom row; print spaces everywhere else to create a hollow full pyramid.

Question 12. Hollow Inverted Full Pyramid

Pattern

```
*****
*  *
*  *
*  *
*  *
*  *
```

Algorithm

- Input the number of rows.
- Run the outer loop from rows to 1.
- Print $\text{rows} - i$ spaces.
- Run the inner loop from 1 to $2*i-1$.
- Print $*$ only on the first position, last position, or first row.
- Otherwise, print a space.
- Move to the next line.

Code

```
#include <iostream>
using namespace std;
```

```
int main() {
    int rows;
    cin >> rows;
```

```
    for (int i = rows; i >= 1; i--) {
```

```
if(j == 1 || j == (2*i-1) || i == rows)
```

- $j == 1 \rightarrow$ Prints the left boundary.
- $j == (2*i-1) \rightarrow$ Prints the right boundary.
- $i == rows \rightarrow$ Prints the complete top row.
- Otherwise, only the boundary stars are printed.

else

- Prints spaces inside the pyramid.
- Makes the pyramid hollow.

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print increasing leading spaces, then print `*` on the top row, left boundary, and right boundary; print spaces everywhere else to create a hollow inverted full pyramid.

Question 13. Floyd's Triangle

Pattern

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Algorithm

1. Input the number of rows.
2. Initialize a variable `num = 1`.
3. Run the outer loop from `1` to `rows`.
4. Run the inner loop from `1` to `i`.

```
// Spaces
for (int j = 1; j <= rows - i; j++) {
    cout << " ";
}

// Stars and Inner Spaces
for (int j = 1; j <= (2 * i - 1); j++) {
    if (j == 1 || j == (2 * i - 1) || i == rows) {
        cout << "*";
    } else {
        cout << " ";
    }
}

cout << endl;
}

return 0;
}
```

Condition Explanation

```
for(int i = rows; i >= 1; i--)
```

- Controls the rows.
- $i = rows \rightarrow$ Starts from the top row.
- $i >= 1 \rightarrow$ Continues until the last row.
- $i-- \rightarrow$ Decreases one row in every iteration.

```
for(int j = 1; j <= rows - i; j++)
```

- Prints leading spaces.
- `rows - i` increases every row.
- First row gets `0` spaces.
- Last row gets the maximum spaces.

```
for(int j = 1; j <= (2*i-1); j++)
```

- Prints the positions of the pyramid.
- $2*i-1$ gives an odd number of positions.
- Since i decreases, the width also decreases by `2` every row.

5. Print `num` and increment it after every print.
6. Move to the next line after each row.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    int num = 1;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            cout << num << " ";
            num++;
        }
        cout << endl;
    }

    return 0;
}
```

Condition Explanation

int num = 1;

- Stores the current number to be printed.
- Starts from `1`.
- Increases after every print.

for(int i = 1; i <= rows; i++)

- Controls the rows.
- `i = 1` → First row.
- `i <= rows` → Last row.
- `i++` → Moves to the next row.

for(int j = 1; j <= i; j++)

- Controls how many numbers are printed in each row.
- Row 1 prints `1` number.
- Row 2 prints `2` numbers.
- Row 3 prints `3` numbers.
- Row `i` prints `i` numbers.

cout << num << " ";

- Prints the current value of `num`.
- A space is printed after each number for proper formatting.

num++;

- Increases the number by `1`.
- Ensures the next printed value is the next consecutive integer.

cout << endl;

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print consecutive numbers row by row, increasing the count of numbers in each row by one.

Question 14. Pascal's Triangle

Pattern

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Algorithm

1. Input the number of rows.

- Run the outer loop from 0 to `rows-1`.
- Print `rows-i-1` spaces.
- Initialize `num = 1` for every row.
- Run the inner loop from 0 to `i`.
- Print `num`.
- Update `num` using the formula `num = num * (i - j) / (j + 1)`.
- Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < rows - i - 1; j++) {
            cout << " ";
        }

        int num = 1;

        for (int j = 0; j <= i; j++) {
            cout << num << " ";
            num = num * (i - j) / (j + 1);
        }

        cout << endl;
    }

    return 0;
}
```

Condition Explanation

for(int i = 0; i < rows; i++)

- Controls the rows.
- `i = 0` → First row.
- `i < rows` → Last row.
- `i++` → Moves to the next row.

for(int j = 0; j < rows - i - 1; j++)

- Prints leading spaces.
- `rows-i-1` decreases every row.
- Aligns the triangle in pyramid form.

int num = 1;

- Every row starts with 1.
- It stores the current value to be printed.

for(int j = 0; j <= i; j++)

- Prints the numbers of the current row.
- Row 0 prints 1 value.
- Row 1 prints 2 values.
- Row 2 prints 3 values.
- Row `i` prints `i+1` values.

num = num * (i - j) / (j + 1);

- Calculates the next value without using factorials.
- Uses the previous number to generate the next one.
- Formula comes from the binomial coefficient:

$$\binom{i}{j} = \frac{i!}{j!(i-j)!}$$

Example for Row 4:

```
1
1 x 4 / 1 = 4
4 x 3 / 2 = 6
6 x 2 / 3 = 4
4 x 1 / 4 = 1
```

Output:

```
1 4 6 4 1
```

cout << endl;

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Start every row with 1 and calculate each next number using the previous number with the binomial coefficient formula.

```
        cout << "0 ";
    }
    cout << endl;
}
return 0;
}
```

Question 15. 0–1 Triangle

Pattern

1
0 1
1 0 1
0 1 0 1
1 0 1 0 1

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to rows.
3. Run the inner loop from 1 to i.
4. Check whether i + j is even or odd.
5. If even, print 1; otherwise, print 0.
6. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;
    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= i; j++) {
            if ((i + j) % 2 == 0)
                cout << "1 ";
            else
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- Controls the rows.
- $i = 1 \rightarrow$ First row.
- $i \leq rows \rightarrow$ Last row.
- $i++ \rightarrow$ Moves to the next row.

```
for(int j = 1; j <= i; j++)
```

- Controls how many values are printed.
- Row 1 prints 1 value.
- Row 2 prints 2 values.
- Row 3 prints 3 values.
- Row i prints i values.

```
if((i + j) % 2 == 0)
```

- Adds the row number and column number.
- % 2 checks whether the sum is even or odd.
- If the sum is even, print 1.
- If the sum is odd, print 0.

Example:

```
(i,j) = (1,1) → 1+1=2 → Even → 1
(i,j) = (2,1) → 2+1=3 → Odd → 0
(i,j) = (2,2) → 2+2=4 → Even → 1
(i,j) = (3,2) → 3+2=5 → Odd → 0

cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print 1 when the sum of the row and column numbers is even; otherwise, print 0.

Question 16. Number Pyramid

Pattern

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to rows.
3. Print rows - i spaces.
4. Run the inner loop from 1 to i.
5. Print the current row number (i) in each iteration.
6. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= rows - i; j++) {
            cout << " ";
        }
    }
}
```

```
for (int j = 1; j <= i; j++) {
    cout << i << " ";
}

    cout << endl;
}

return 0;
}
```

Condition Explanation

for(int i = 1; i <= rows; i++)

- Controls the rows.
- $i = 1 \rightarrow$ First row.
- $i \leq rows \rightarrow$ Last row.
- $i++ \rightarrow$ Moves to the next row.

for(int j = 1; j <= rows - i; j++)

- Prints leading spaces.
- $rows - i$ decreases every row.
- First row gets the maximum spaces.
- Last row gets 0 spaces.

for(int j = 1; j <= i; j++)

- Prints numbers in the current row.
- Row 1 prints 1 number.
- Row 2 prints 2 numbers.
- Row 3 prints 3 numbers.
- Row i prints i numbers.

cout << j << " ";

- Prints the current row number.
- The same number is printed throughout the row.
- A space is added after each number for proper formatting.

cout << endl;

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print leading spaces, then print the current row number (i) exactly i times.

Question 17. Number Palindrome Pyramid

Pattern

```
1
212
32123
4321234
543212345
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to rows.
3. Print rows - i spaces.
4. Print numbers from i down to 1.
5. Print numbers from 2 up to i.
6. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = 1; i <= rows; i++) {
        // Spaces
        for (int j = 1; j <= rows - i; j++) {
            cout << " ",
```

```
        }
    }

    // Decreasing Numbers
    for (int j = i; j >= 1; j--) {
        cout << j;
    }

    // Increasing Numbers
    for (int j = 2; j <= i; j++) {
        cout << j;
    }

    cout << endl;
}

return 0;
}
```

Condition Explanation

for(int i = 1; i <= rows; i++)

- Controls the rows.
- $i = 1 \rightarrow$ First row.
- $i <= \text{rows} \rightarrow$ Last row.
- $i++ \rightarrow$ Moves to the next row.

for(int j = 1; j <= rows - i; j++)

- Prints leading spaces.
- $\text{rows} - i$ decreases every row.
- First row gets maximum spaces.
- Last row gets 0 spaces.

for(int j = i; j >= 1; j--)

- Prints numbers in decreasing order.
- Starts from the current row number.
- Ends at 1.

Example for Row 4:

4321

```
for(int j = 2; j <= i; j++)
```

- Prints numbers in increasing order.
- Starts from 2 because 1 is already printed in the previous loop.
- Ends at the current row number.

Example for Row 4:

234

Together, both loops produce:

4321234

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

Complexity

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

Logic in One Line

Print leading spaces, then print numbers in decreasing order to 1, followed by increasing numbers back to the current row to form a palindrome.

Question 18. Character Pyramid

Pattern

```
A
AB
ABC
ABCD
ABCDE
```

Algorithm

1. Input the number of rows.
2. Run the outer loop from 1 to rows.
3. Print rows - i spaces.

4. Initialize a character variable `ch = 'A'`.
5. Run the inner loop from 1 to i.
6. Print `ch` and increment it.
7. Move to the next line.

Code

```
#include <iostream>
using namespace std;

int main() {
    int rows;
    cin >> rows;

    for (int i = 1; i <= rows; i++) {
        // Spaces
        for (int j = 1; j <= rows - i; j++) {
            cout << " ";
        }

        // Characters
        char ch = 'A';

        for (int j = 1; j <= i; j++) {
            cout << ch << " ";
            ch++;
        }

        cout << endl;
    }

    return 0;
}
```

Condition Explanation

```
for(int i = 1; i <= rows; i++)
```

- Controls the rows.
- `i = 1` → First row.
- `i <= rows` → Last row.
- `i++` → Moves to the next row.

```
for(int j = 1; j <= rows - i; j++)
```

Print leading spaces, then print characters starting from **A**, increasing one alphabet at a time in every row.

Question 19. Butterfly Pattern

Pattern

```
* *
** **
*** ***
**** ****
***** *****
***** *****
**** ****
*** ***
** **
* *
```

Algorithm

1. Input the number of rows (**n**).
2. Print the **upper half** from **1** to **n**.
3. Print left stars (**i** stars).
4. Print middle spaces (**2*(n-i)** spaces).
5. Print right stars (**i** stars).
6. Print the **lower half** from **n-1** to **1**.
7. Repeat the same steps for the lower half.
8. Move to the next line after each row.

Code

```
#include <iostream>
using namespace std;

int main() {

    int n;
    cin >> n;

    // Upper Half
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << " * ";

        for (int j = 1; j <= 2 * (n - i); j++)
            cout << " ";
    }
```

- Prints leading spaces.
 - **rows** - **i** decreases every row.
 - First row gets maximum spaces.
 - Last row gets **0** spaces.
- char ch = 'A';
- Initializes the character as **A**.
 - This statement is inside the outer loop, so every row starts from **A**.

```
for(int j = 1; j <= i; j++)
```

- Prints characters in the current row.
 - Row 1 prints 1 character.
 - Row 2 prints 2 characters.
 - Row 3 prints 3 characters.
 - Row **i** prints **i** characters.
- ```
cout << ch << " ";

 Prints the current character.
 A space is printed after every character for better formatting.

ch++;
```

- Moves to the next alphabet.
- **A** → **B** → **C** → **D** → **E** ...

Example for Row 4:

A B C D

```
cout << endl;
```

Moves the cursor to the next line after completing the current row.

**Complexity**

- Time Complexity: **O(n<sup>2</sup>)**
- Space Complexity: **O(1)**

**Logic in One Line**

```
for (int j = 1; j <= i; j++)
 cout << "n",
```

```
 cout << endl;
```

```
}
```

```
// Lower Half
```

```
for (int i = n - 1; i >= 1; i--) {
```

```
 for (int j = 1; j <= i; j++)
```

```
 cout << "n",
```

```
 for (int j = 1; j <= 2 * (n - i); j++)
```

```
 cout << "n",
```

```
 for (int j = 1; j <= i; j++)
```

```
 cout << "n",
```

```
 cout << endl;
```

```
 }
```

```
 return 0;
```

```
 }
```

### Condition Explanation

```
for(int i = 1; i <= n; i++)
```

- Prints the upper half.
- Starts with **1** star on each side.
- Increases one star every row.

```
for(int j = 1; j <= i; j++)
```

- Prints the left wing.
- Number of stars increases every row.

```
for(int j = 1; j <= 2*(n-i); j++)
```

- Prints the middle spaces.
- **2\*(n-1)** because there are two gaps shrinking together.
- Spaces decrease by **2** every row.
- Last row gets **0** spaces.

```
for(int j = 1; j <= i; j++)
```

- Prints the right wing.
- Same number of stars as the left wing.

```
for(int i = n-1; i >= 1; i--)
```

- Prints the lower half.

- Starts from **n-1** to avoid printing the middle row twice.
- Stars decrease one row at a time.
- Spaces increase accordingly.

```
 cout << endl;
```

Moves the cursor to the next line after completing the current row.

### Complexity

- Time Complexity:  **$O(n^2)$**
- Space Complexity:  **$O(1)$**

### Logic in One Line

Print increasing stars on both sides with decreasing middle spaces for the upper half, then reverse the process to form a butterfly.